



Builder

| Separate constructing a complex object from its representation — so one process can build many forms.

 DIVE 01 · 10 km
Airplane porthole
The big picture & contours

→

 DIVE 02 · 1 km
Train window
Theses, examples, terms

→

 DIVE 03 · 100 m
Park walk
How the pieces relate

→

 DIVE 04 · 1 cm
Microscope
Exact wording & gotchas

 Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

Builder splits how a complex object is assembled — from what it finally is, so one build sequence can yield many representations.

— THE CONTOURS

- **Build step by step**
assemble a complex object across many small calls

- **Same steps, many products**
one Director drives different ConcreteBuilders

- **No telescoping constructors**
named steps replace unreadable 8-argument ctors

— USE IT WHEN — GoF applicability

 **Construction is complex**
the assembly algorithm is independent of the parts and how they're combined

 **Same process, different reps**
the build must be able to produce different representations of the product

 Creational family: Factory Method · Abstract Factory · Builder · Prototype · Singleton — Builder focuses on step-by-step assembly of one complex product, where Abstract Factory returns families of simpler ones.

↓ DIVE DEEPER

 Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

— THE THREE PARTS

Builder
abstract interface — one method per part (buildA / buildB)

ConcreteBuilder
assembles parts, tracks the result, exposes getResult()

Director
knows the recipe — calls the build steps in the right order

— IN THE WILD

 **java.lang.StringBuilder**
append() steps, then a final toString()

 **Lombok @Builder**
generates a fluent builder for value objects

 **SQL query builders**
Knex / jOOQ assemble a query part by part

 **RequestBuilder / fetch**
set method, headers, body — then build()

<> Director.construct() → Builder.buildPartA / B / C() → ConcreteBuilder.getResult() returns the Product.

| **GoF intent:** “Separate the construction of a complex object from its representation so that the same construction process can create different representations.”

↓ DIVE DEEPER

 Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Director
knows the steps

→

buildA / B / C()
add the parts

→

ConcreteBuilder
tracks the result

→

✓ getResult()
the product

— WHY IT FOLLOWS

→ **Steps live in the Director** ⇒ the construction sequence is written once and reused for any builder

→ **Parts live in the Builder** ⇒ swap the ConcreteBuilder to change the representation

→ **Result fetched at the end** ⇒ the product stays hidden and incomplete until build() is called

 A Builder often builds a Composite tree; Abstract Factory also hides concrete classes but returns its product immediately, while Builder returns it as a final step.

— STRUCTURE · UML

Builder

«interface»

+ buildPartA(): void
+ buildPartB(): void
+ getResult(): Product

The Director holds a Builder and calls the steps in order; a ConcreteBuilder implements them and returns the finished Product via getResult().

— WHAT IT BUYS YOU

✓ **Varies the internal rep** — product internals stay hidden behind the Builder interface

✓ **Isolates construction code** — assembly logic is separate from representation

✓ **Finer control of building** — the object is built step by step, under the Director's control

✓ **Reuses the build process** — one Director can construct many different representations

✓ **Encapsulates assembly** — the client never sees the parts being put together

↓ DIVE DEEPER

 Microscope

DIVE 04 · DETAILS

▼ 1 cm

| **Definition:** “Separate the construction of a complex object from its representation so that the same construction process can create different representations.”
— GoF, “Design Patterns”, 1994

— THE CODE

```
class UrlBuilder {
  private p: string[] = [];
  add(seg: string) { this.p.push(seg); return this; }
  query(k, v) { /* ..append ?k=v */ return this; }
  build(): string { return this.p.join("/"); }
}

// chain the steps → one terminal build():
const url = new UrlBuilder()
  .add("users").add(id).query("tab", "x")
  .build();
```

— COMPARE THE ALTERNATIVES

Property	Builder	Factory IM.	Abstract F.	Prototype
Builds step by step	✓	✗	✗	✗
Hides representation	✓	~	✓	~
Good for many optionals	✓	✗	✗	✗
Returns product at the end	✓	✗	✗	✗
Same steps, many reps	✓	✗	~	✗

Best fit → Builder: complex stepwise construction · Factory Method: one product by subclass · Abstract Factory: matching families · Prototype: clone. (“~” = partial.)

— INTERVIEW PITFALLS

Q “Builder vs Factory?”
A factory returns a product in one call; a Builder assembles it over several steps and hands it back at the end — ideal when there are many optional parts.

Q “Where does the Director fit?”
It owns the construction sequence. Modern fluent builders usually inline the recipe and skip the Director entirely.

Q “Why return this?”
To chain steps fluently. The terminal build() / getResult() returns the finished product.

Q “Builder vs telescoping ctor?”
Named steps replace many overloaded constr uctors, so optional parameters stay readable and order-independent.

— VARIANTS

Fluent / chained
each step returns this — the everyday JS / Java form

Step builder
types force a required call order at compile time

Immutable build()
collects fields, freezes the product at the end

Director-driven
GoF classic — a Director encodes the build recipe

Telescoping killer
replaces many-argument constructors with named steps

Functional
an options object + factory fn — no builder class (JS)

— NUANCES & GOTCHAS

• **The Director is optional** — modern fluent builders fold the recipe into the call chain and drop the Director

• **Return this to chain** — every step returns the builder; build() returns the product

• **vs Abstract Factory** — Builder returns its product last, in steps; Abstract Factory returns it in one call

• **Validate in build()** — enforce required fields and invariants once, at the terminal step

• **Mutable builder, frozen product** — the common shape: assemble freely, then emit an immutable value

• **Don't overuse** — a two-field object needs no builder; a literal is clearer

— WATCH OUT

 **Anti-pattern:** Skip it for simple objects — a constructor or a plain literal is clearer. Builders pay off only with many optional parts, ordering rules, or several representations.

 **Modern take:** in TS a typed fluent builder (or just an options object) covers most cases; reach for a Director only when one recipe must drive several builders.

• Vasyi Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994) **v1**